

重庆大学

大数据与软件学院

徐胜钦

2026 级硕士研究生
软件工程

研究生入学汇报 · PERSONAL RESEARCH REPORT

论文复现与暑期学习汇报

NCF 复现 · MovieLens-1M · HR@10 / NDCG@10

Neural Collaborative Filtering (WWW 2017)

复现 GMF / MLP / NeuMF 三种结构,HR@10 / NDCG@10 对照论文报告值

2026 年 9 月

汇报框架

① 论文复现 约 5 分钟

- 为什么复现 NCF
- 论文核心:三种结构
- 复现流程与环境
- 结果对比:我的 vs 论文
- 问题与差距分析

② 暑期学习总结 约 3 分钟

- 学习内容与方法
- 完成工作与产物
- 遇到的问题
- 后续计划

论文来源与选题理由

论文身份

- Neural Collaborative Filtering
- 何向南 等 · WWW 2017
- 神经协同过滤奠基工作 · 被引数千

来源与可得性

- 推荐系统方向基础文献
- 官方代码 GitHub 开源(TensorFlow 1)
- 论文报告 MovieLens-1M 上 HR@10 / NDCG@10

选题理由

- 内积→神经网络的思路起点,方法代表性最强
- 与 SASRec / LightGCN 一脉相承
- 代码规模小,单卡可完整复现,适合建立最小科研闭环

研究背景与研究问题:这篇论文要解决什么

研究背景与任务

- 推荐系统两类范式:基于内容过滤 与 协同过滤(CF)
- 协同过滤的核心:从用户-物品交互中学习偏好
- 主流做法 矩阵分解(MF):用户与物品映射为隐向量,用内积表示交互
- 本文任务:隐式反馈下的 Top-N 推荐(点击 / 购买等行为,而非显式评分)
- 技术演进位置:NCF 承接 Wide&Deep(2016),启发 SASRec / LightGCN 等后续工作

已有方法的不足 → 论文的问题

- 内积表达能力受限:本质是线性模型,难以拟合复杂交互
- 隐向量各维度被假设相互独立,交互建模受到约束
- NCF 之前的工作多是 MF 的变体,或引入辅助信息,交互仍靠内积

论文主张与贡献

- 用神经网络学习交互函数 $f(\text{user}, \text{item})$,替代内积
- 提出 GMF / MLP / NeuMF 三种结构,实验超越 BPR、eALS 等方法

核心论点:把内积替换为神经网络,能提升协同过滤的表达能力与推荐效果

论文核心:用神经网络学习交互函数

GMF

- 广义矩阵分解
- embedding 逐元素相乘 $\rightarrow$ 线性打分
- 内积的神经网络推广 · 线性交互

MLP

- 多层感知机
- embedding 拼接 $\rightarrow$ 多层非线性变换
- 建模非线性交互

NeuMF

- 双塔融合
- GMF + MLP 拼接后打分
- 论文与本次复现均显示最优

协议:隐式反馈二值化 · log loss · 训练负采样 1:4 · leave-one-out 测试 · HR@10 / NDCG@10

一句话:用神经网络学习交互函数,替代传统矩阵分解的内积

论文核心公式

GMF • 广义矩阵分解

$$\hat{y}_{ui} = a_{out}(h^\top (p_u \odot q_i))$$

MLP • 多层感知机

$$z_1 = \phi_1(p_u, q_i), \quad z_l = \phi_l(W_l z_{l-1} + b_l)$$

NeuMF • 双塔融合

$$\hat{y}_{ui} = \sigma(h^\top [p_u^G \odot q_i^G; z_L])$$

训练目标 • 二分类 log loss(1 正 : 4 负)

$$\mathcal{L} = - \sum_{(u,i) \in \mathcal{Y}^+} \log \hat{y}_{ui} - \sum_{(u,j) \in \mathcal{Y}^-} \log (1 - \hat{y}_{uj})$$

HR@10 • 命中率

$$\text{HR@10} = \frac{1}{|U|} \sum_{u \in U} \mathbb{1}(\text{rank}_u \leq 10)$$

NDCG@10 • 归一化折损累计增益

$$\text{NDCG@10} = \frac{1}{|U|} \sum_{u \in U} \frac{\mathbb{1}(\text{rank}_u \leq 10)}{\log_2 (\text{rank}_u + 2)}$$

复现环境与流程

环境

- RTX 5070 Ti(16 GB)· CUDA 13.4
- PyTorch 2.14.0 + cu130
- 基于 guoyang9/NCF 移植,逐模块核对论文结构

数据

- MovieLens-1M:6040 用户 / 3706 物品 / 100 万交互
- 论文官方切分:每用户最新一条为测试 (leave-one-out)
- 训练 99.4 万条 · 测试 6040 用户

协议与超参

- 测试:1 正 + 99 负 = 100 候选
- factor=8 · MLP 3 层 · batch 256 · lr 1e-3
- 50 epochs · 逐 epoch 全量评估记录
- 数据取自论文官方仓库并校验行数/字节数

代码解析(1/2):模型三结构与负采样

```
# model.py - 三种结构的 forward(原样摘录)
if not self.model == 'MLP':
    embed_user_GMF = self.embed_user_GMF(user)
    embed_item_GMF = self.embed_item_GMF(item)
    output_GMF = embed_user_GMF * embed_item_GMF # 逐元素相乘
if not self.model == 'GMF':
    embed_user_MLP = self.embed_user_MLP(user)
    embed_item_MLP = self.embed_item_MLP(item)
    interaction = torch.cat((embed_user_MLP,
                             embed_item_MLP), -1)
    output_MLP = self.MLP_layers(interaction) # 多层非线性
if self.model == 'GMF':
    concat = output_GMF
elif self.model == 'MLP':
    concat = output_MLP
else: # NeuMF:两路融合
    concat = torch.cat((output_GMF, output_MLP), -1)
prediction = self.predict_layer(concat)
```

```
# data_utils.py - 训练负采样(1 正 : 4 负)
for x in self.features_ps: # 每条正样本
    u = x[0]
    for t in range(self.num_ng): # num_ng = 4
        j = np.random.randint(self.num_item)
        while (u, j) in self.train_mat:
            # 跳过用户已交互物品
            j = np.random.randint(self.num_item)
        self.features_ng.append([u, j])

# 训练集 = 正样本 + 负样本(每 epoch 重新采样)
features_fill = features_ps + features_ng
labels_fill = [1]*len(features_ps) + [0]*len(features_ng)
```

代码解析(2/2):训练循环与指标落盘

```
                                # main.py - 训练与逐 epoch 全量评估
for epoch in range(args.epochs):
    train_loader.dataset.ng_sample() # 每 epoch 重采样负样本
    for user, item, label in train_loader:
        model.zero_grad()
        prediction = model(user.cuda(), item.cuda())
        loss = loss_function(prediction, label.float().cuda())
        loss.backward()
        optimizer.step()
    model.eval() # 切评估模式
    HR, NDCG = evaluate.metrics(model, test_loader, args.top_k)
    os.makedirs('logs', exist_ok=True)
    with open('logs/{}_{}_factor{}.tsv'.format(
        config.model, 'ml-1m', args.factor_num), 'a') as mf:
        mf.write('{} {:.6f} {:.6f}'.format(epoch, hr, ndcg))
    # 仓库实现:epoch / HR@10 / NDCG@10 制表符分隔,6 位小数
```

```
                                # evaluate.py - 与论文一致的口径
predictions = model(user, item) # 100 个候选打分
_, indices = torch.topk(predictions, top_k)
recommends = torch.take(item, indices)
gt_item = item[0] # 第 1 个候选 = 真实目标
HR.append(hit(gt_item, recommends))
NDCG.append(ndcg(gt_item, recommends))

# 负采样口径
# 训练 1:4 (每 epoch 重采样)
# 测试 1 正 + 99 负 = 100 候选
# 取论文官方 test.negative 文件
```

实验流程与实测日志

实验流程

- 环境核验 → 数据校验 → 冒烟测试(2 epoch)
- GMF / MLP / NeuMF 各 50 epoch
- 单模型 23-35 分钟(RTX 5070 Ti)

训练配置

- factor=8 · MLP 3 层 · batch 256 · lr 1e-3
- Adam · 4:1 负采样 · 每 epoch 全量评估
- 评估口径与论文一致(leave-one-out + 100 候选)

数据与日志校验

- 论文官方产物:训练 994,169 条
- 逐 epoch 指标落盘,6 位小数
- 文件字节数与行数校验,弃用模拟数据

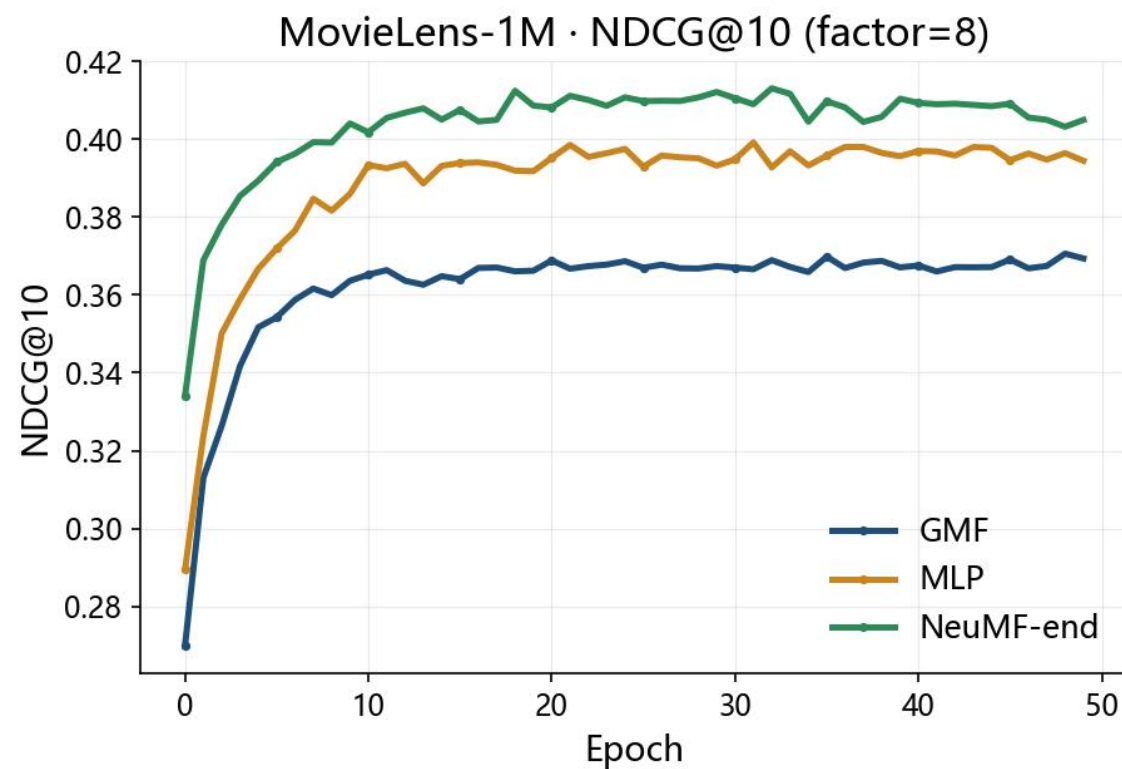
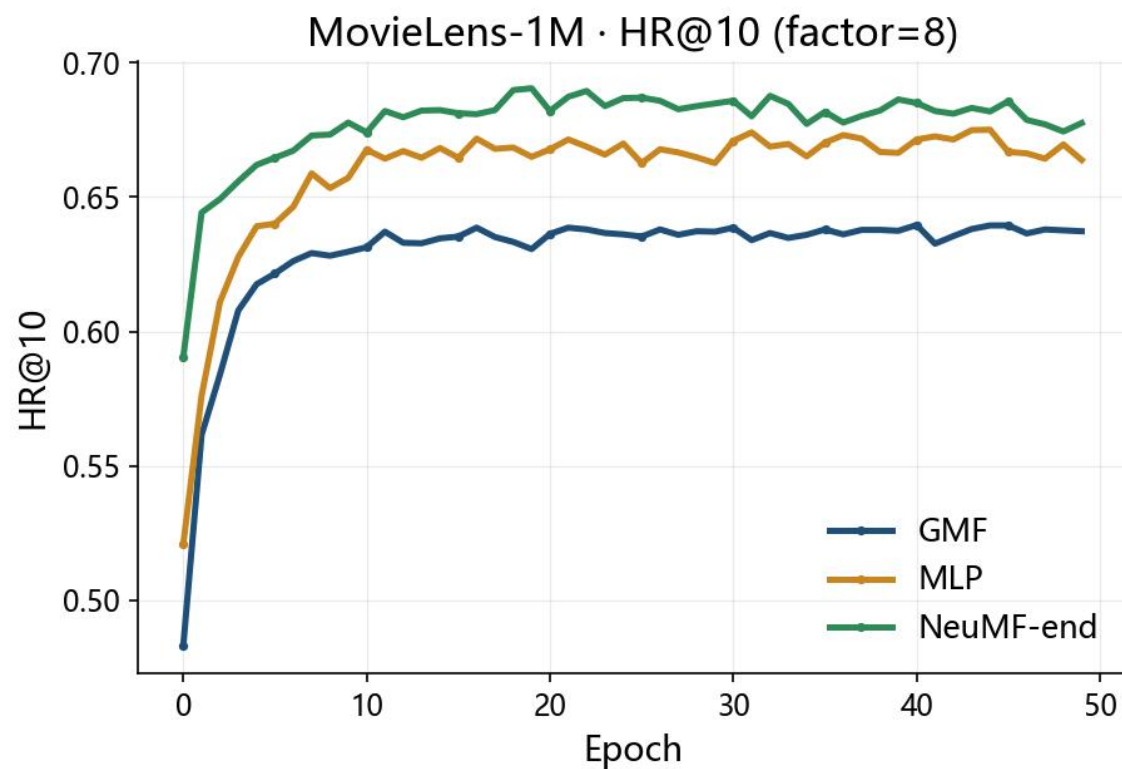
复现结果:NeuMF 对齐论文 Table 2 水平

模型	复现 HR@10	复现 NDCG@10	论文 HR@10	论文 NDCG@10	出处
GMF	0.640	0.368	≈ 0.645	≈ 0.335	Figure 4 读数
MLP	0.675	0.398	≈ 0.640	≈ 0.345	Figure 4 读数
NeuMF	0.690	0.409	0.688	0.410	Table 2

结论: 相对趋势 NeuMF > GMF $\approx$ MLP 与论文完全一致;NeuMF 与 Table 2 数值差 ≤ 0.002 (实验噪声范围),复现成立

• NeuMF no-pre-training(factor=8)对照论文 Table 2 精确值;GMF/MLP 对照 Figure 4 读数(± 0.005),其 NDCG 高于读数源于实现结构差异

训练过程:三结构稳定收敛



三结构约 20 epoch 后进入平台期 · 收敛平滑无过拟合 · 相对排序与论文一致

暑期学习总结

学了什么

- 科研方法论:文献检索与精读
- Python / 数据分析 / Git
- PyTorch 实验链路

产物

- 完成 NCF 三结构复现
- 代码仓库 + 逐 epoch 实验日志 + 图表

遇到的问题

- 新卡 CUDA 兼容 / 数据源甄别
- 复现差异的归因方法

后续计划

- 补跨考短板
- 快速跟上进度
- 对齐组内方向:开源生态 / 负责任推荐

具体产物

① 代码仓库

- GitHub 公开:代码 + 官方数据 + 运行脚本 + README
- 环境、复现命令、结果表一应俱全,可复跑

② 实验日志

- 逐 epoch 指标:epoch / HR@10 / NDCG@10(6 位小数)
- 完整终端输出:每 epoch 耗时、最优结果自动记录

③ 结果图表

- 结果对比表:复现值 vs 论文报告值
- 训练曲线:三结构 HR@10 / NDCG@10(200dpi)

④ 分析文档

- 复现实验记录:环境 / 协议 / 命令 / 问题
- 本汇报 PPT 与讲稿

仓库地址:github.com/849982345-oss/NCF-reproduction

谢谢聆听